

0.18.1. Avancer avec `deplace_x` — de base, ailleurs

Le 0.18 sur la ligne 8, avec le B.

1. Ce qu'il faut comprendre

C'est le **0.18**, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **ligne 8** au lieu de 0 (2e réglage de `deplace_x`).

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.18.2 met les deux ensemble.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1596 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

2. UNE VARIABLE — X

```
uint8_t x = 0;           // la colonne du B
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Les changements : la ligne (0 → 8) et la lettre (ALPHABET[1], le B).
    x = deplace_x(x, 8, ALPHABET[1], pas);    // x vaut 5

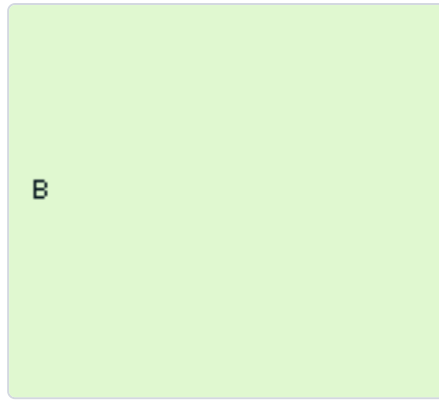
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

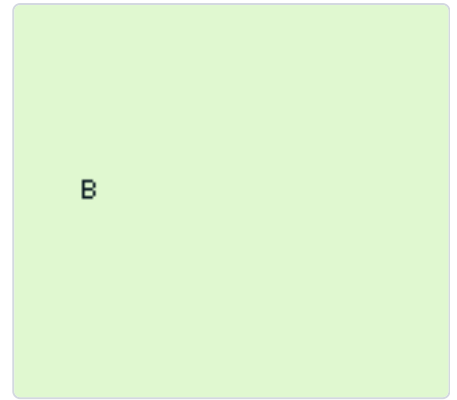
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



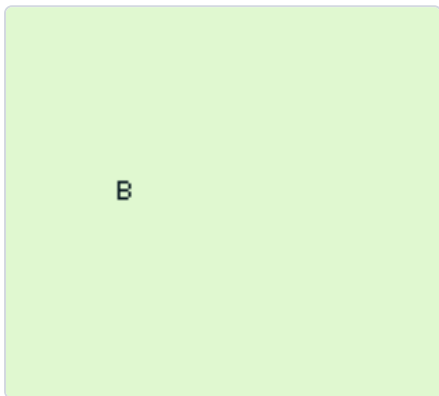
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un B qui avance de 5 cases sur la ligne 8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B finit en (5, 8) (5,8)

✓ x vaut 5

Ensuite : **0.18.2. Avancer avec `deplace_x` — doublé, deux positions** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++